Mars 모듈 단위 테스트 분석

개요

이 문서는 Mars 모듈의 단위 테스트 파일들을 분석하여 테스트 구조, 헬퍼 함수, 목 객체 사용법, 그리고 테스트 시나리오와 검증 로직을 상세히 설명합니다.

테스트 파일 구조

Mars 모듈의 단위 테스트는 다음과 같이 구성되어 있습니다:

```
x/mars/
├── keeper/
| ├── keeper_test.go # Keeper 테스트 환경 설정 및 공통 헬퍼
| ├── msg_update_params_test.go # 메시지 처리 테스트
| ├── query_params_test.go # 쿼리 처리 테스트
| └── genesis_test.go # 제네시스 상태 처리 테스트
└── types/
└── genesis_test.go # 제네시스 상태 검증 테스트
```

테스트 헬퍼 함수 및 유틸리티

1. fixture 구조체

`keeper_test.go` 에서 정의된 `fixture` 구조체는 모든 테스트에서 공통으로 사용되는 테스트 환경을 캡슐화합니다:

```
// fixture는 테스트에 필요한 공통 컴포넌트들을 담는 구조체입니다.
// 각 테스트에서 반복적으로 사용되는 컨텍스트, keeper, 코덱 등을 포함합니다.
type fixture struct {
// ctx는 테스트 실행을 위한 컨텍스트입니다.
// 블록체인 상태와 트랜잭션 정보를 포함합니다.
ctx context.Context

// keeper는 테스트 대상인 mars 모듈의 Keeper 인스턴스입니다.
keeper keeper.Keeper

// addressCodec는 주소 인코딩/디코딩을 위한 코덱입니다.
// 테스트에서 주소 변환이 필요할 때 사용됩니다.
addressCodec address.Codec
}
```

2. initFixture() 헬퍼 함수

모든 테스트에서 사용되는 핵심 헬퍼 함수로, 완전한 테스트 환경을 초기화합니다:

```
// initFixture는 테스트를 위한 환경을 초기화하고 fixture를 생성합니다.
// 이 함수는 각 테스트에서 공통으로 사용되는 설정을 담당합니다.
```

```go
func initFixture(t *testing.T) *fixture
```

주요 초기화 과정:

1. **테스트 헬퍼 표시**: `t.Helper()` 호출로 테스트 실패 시 정확한 위치 표시
2. **인코딩 설정**: `moduletestutil.MakeTestEncodingConfig()` 사용
3. **주소 코덱 생성**: Bech32 형식의 주소 처리를 위한 코덱 설정
4. **저장소 키 생성**: 테스트용 KV 저장소 키 생성
5. **저장소 서비스 생성**: 실제 블록체인과 동일한 저장소 인터페이스 제공
6. **테스트 컨텍스트 생성**: 메모리 데이터베이스와 임시 저장소 포함
7. **권한 설정**: 거버넌스 모듈 주소를 authority로 설정
8. **Keeper 인스턴스 생성**: 모든 의존성을 주입하여 완전한 keeper 생성
9. **기본 파라미터 초기화**: 테스트 시작 전 기본값 설정

3. 테스트 유틸리티 (testutil/sample)

`testutil/sample/sample.go` 에서 제공하는 헬퍼 함수:

```go
// AccAddress returns a sample account address
// 테스트에서 사용할 샘플 계정 주소를 생성합니다.
func AccAddress() string {
pk := ed25519.GenPrivKey().PubKey() // 임의의 개인키 생성
addr := pk.Address() // 공개키에서 주소 추출
return sdk.AccAddress(addr).String() // SDK 주소 형식으로 변환
}
```

목 객체 및 모의 환경

Mars 모듈의 테스트는 실제 목 객체 라이브러리를 사용하지 않고, 대신 Cosmos SDK의 테스트 유틸리티를 활용하여 모의 환경을 구성합니다:

1. 모의 저장소

- `testutil.DefaultContextWithDB()` : 메모리 기반 데이터베이스 제공
- `storetypes.NewKVStoreKey()` : 테스트용 저장소 키 생성
- `runtime.NewKVStoreService()` : 실제 환경과 동일한 저장소 인터페이스

2. 모의 코덱

- `moduletestutil.MakeTestEncodingConfig()` : 테스트용 인코딩 설정
- 실제 운영 환경과 동일한 직렬화/역직렬화 로직 제공

3. 모의 주소 시스템

- `addresscodec.NewBech32Codec()` : Bech32 주소 코덱
- `authtypes.NewModuleAddress()` : 모듈 주소 생성

테스트 시나리오 및 검증 로직

1. 메시지 처리 테스트 (msg_update_params_test.go)

테스트 대상: `MsgUpdateParams` 메시지 처리 로직

테스트 시나리오:

```
testCases := []struct {
name string // 테스트 케이스 이름
```

```
input *types.MsgUpdateParams // 입력 메시지

expErr bool // 에러 발생 예상 여부

expErrMsg string // 예상 에러 메시지

}{

{

// 시나리오 1: 잘못된 권한 주소

name: "invalid authority",

input: &types.MsgUpdateParams{

Authority: "invalid", // 잘못된 형식의 주소

Params: params,

},

expErr: true,

expErrMsg: "invalid authority",

},

{

// 시나리오 2: 빈 파라미터로 업데이트

name: "send enabled param",

input: &types.MsgUpdateParams{

Authority: authorityStr, // 올바른 권한 주소

Params: types.Params{}, // 빈 파라미터

},

expErr: false,

},
```

```
{
// 시나리오 3: 정상적인 파라미터 업데이트
name: "all good",
input: &types.MsgUpdateParams{
Authority: authorityStr, // 올바른 권한 주소
Params: params, // 유효한 파라미터
},
expErr: false,
},
}
```

검증 로직:

- **권한 검증**: 올바른 authority 주소인지 확인
- **에러 처리**: 예상된 에러 메시지가 포함되는지 검증
- **성공 케이스**: 에러 없이 처리되는지 확인

2. 쿼리 처리 테스트 (query_params_test.go)

테스트 대상: 파라미터 조회 쿼리

테스트 시나리오:

1. **환경 준비**: 기본 파라미터를 저장소에 설정
2. **쿼리 실행**: `QueryParamsRequest` 로 파라미터 조회
3. **결과 검증**: 저장된 값과 조회된 값의 일치성 확인

검증 로직:

```go
// 1. 에러가 발생하지 않았는지 확인
require.NoError(t, err, "파라미터 조회 중 에러가 발생했습니다: %v", err)

// 2. 응답 내용이 예상과 정확히 일치하는지 확인
require.Equal(t, &types.QueryParamsResponse{Params: params}, response,
"조회된 파라미터가 저장된 파라미터와 일치하지 않습니다")
```

3. 제네시스 처리 테스트 (keeper/genesis_test.go)

테스트 대상: 제네시스 상태 초기화 및 내보내기

테스트 시나리오:

1. **제네시스 상태 생성**: 기본 파라미터를 포함한 초기 상태 정의
2. **초기화 테스트**: `InitGenesis()` 함수로 상태 저장소에 설정
3. **내보내기 테스트**: `ExportGenesis()` 함수로 현재 상태 추출
4. **무결성 검증**: 초기화한 상태와 내보낸 상태의 일치성 확인

검증 로직:

```go
// 제네시스 초기화 검증
err := f.keeper.InitGenesis(f.ctx, genesisState)
require.NoError(t, err, "제네시스 초기화 중 에러가 발생했습니다: %v", err)

// 제네시스 내보내기 검증
got, err := f.keeper.ExportGenesis(f.ctx)
require.NoError(t, err, "제네시스 내보내기 중 에러가 발생했습니다: %v", err)
```

```
require.NotNil(t, got, "내보낸 제네시스 상태가 nil입니다")


// 데이터 무결성 검증
require.EqualExportedValues(t, genesisState.Params, got.Params,
"초기화된 파라미터와 내보낸 파라미터가 일치하지 않습니다")
```

4. 제네시스 검증 테스트 (types/genesis_test.go)

테스트 대상: 제네시스 상태 유효성 검증

테스트 시나리오:

```
tests := []struct {
desc string // 테스트 케이스 설명
genState *types.GenesisState // 테스트할 제네시스 상태
valid bool // 유효성 검증 예상 결과
}{
{
// 기본 제네시스 상태 검증
desc: "default is valid",
genState: types.DefaultGenesis(),
valid: true,
},
{
// 빈 제네시스 상태 검증
```

```
desc: "valid genesis state",
genState: &types.GenesisState{},
valid: true,
},
}
```

검증 로직:

- **테이블 기반 테스트**: 여러 시나리오를 체계적으로 검증
- **유효성 검사**: `Validate()` 메서드를 통한 상태 검증
- **에러 처리**: 유효/무효 상태에 따른 적절한 에러 처리 확인

테스트 패턴 및 모범 사례

1. 테이블 기반 테스트 (Table-Driven Tests)

- 여러 시나리오를 구조화된 방식으로 테스트
- 코드 중복 최소화 및 가독성 향상
- 새로운 테스트 케이스 추가 용이성

2. 서브테스트 활용

- `t.Run()` 을 사용한 독립적인 테스트 실행
- 각 테스트 케이스의 격리된 환경 보장
- 실패 시 정확한 테스트 케이스 식별 가능

3. 헬퍼 함수 패턴

- `t.Helper()` 사용으로 정확한 실패 위치 표시
- 공통 설정 로직의 재사용성 향상
- 테스트 코드의 가독성 및 유지보수성 개선

4. 어설션 라이브러리 활용

- `github.com/stretchr/testify/require` 사용
- 명확한 에러 메시지와 함께 테스트 실패 원인 제공
- 다양한 검증 함수로 정확한 테스트 수행

테스트 커버리지 및 검증 범위

1. 기능별 커버리지

- **Keeper 기능**: 파라미터 저장/조회, 권한 검증
- **메시지 처리**: 입력 검증, 권한 확인, 상태 업데이트
- **쿼리 처리**: 데이터 조회, 응답 형식 검증
- **제네시스 처리**: 초기화, 내보내기, 상태 검증

2. 에러 시나리오 테스트

- 잘못된 입력에 대한 적절한 에러 처리
- 권한 없는 접근에 대한 보안 검증
- 예상 에러 메시지의 정확성 확인

3. 데이터 무결성 검증

- 저장된 데이터와 조회된 데이터의 일치성
- 제네시스 초기화와 내보내기의 일관성

- 상태 변경 전후의 데이터 정확성

이러한 포괄적인 테스트 구조를 통해 Mars 모듈의 안정성과 신뢰성을 보장하며, 향후 코드 변경 시에도 기존 기능의 정상 동작을 검증할 수 있습니다.